

Chapter 2 - Architectures

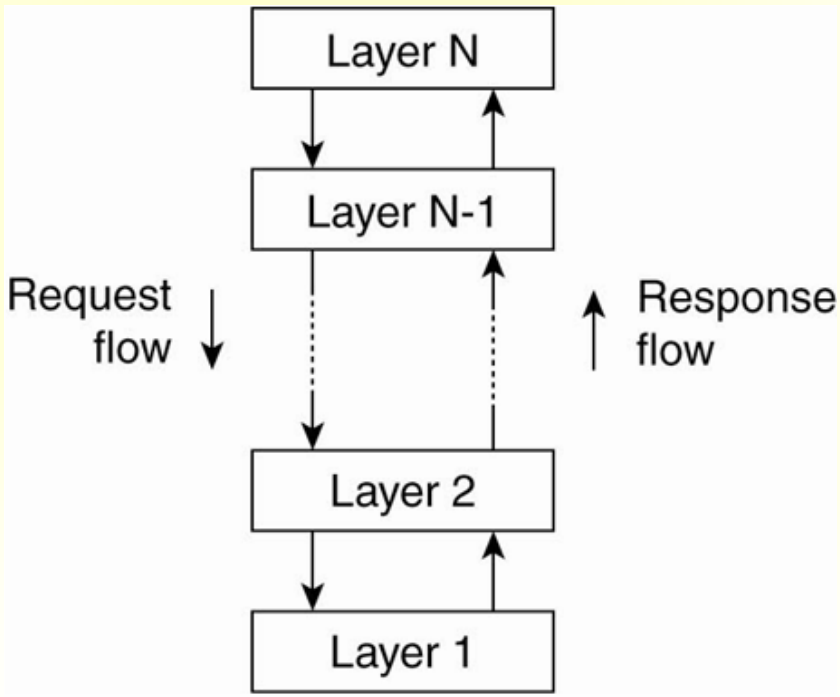
Introduction

- how to organize the collection of software components
- we will discuss
  - logical organization and physical organization
    - i.e., software or system architectures: how they are organized and how they communicate
  - architectural styles
  - system architectures: centralized (client-server) vs decentralized ones (where machines play equal roles)

2.1 Architectural Styles

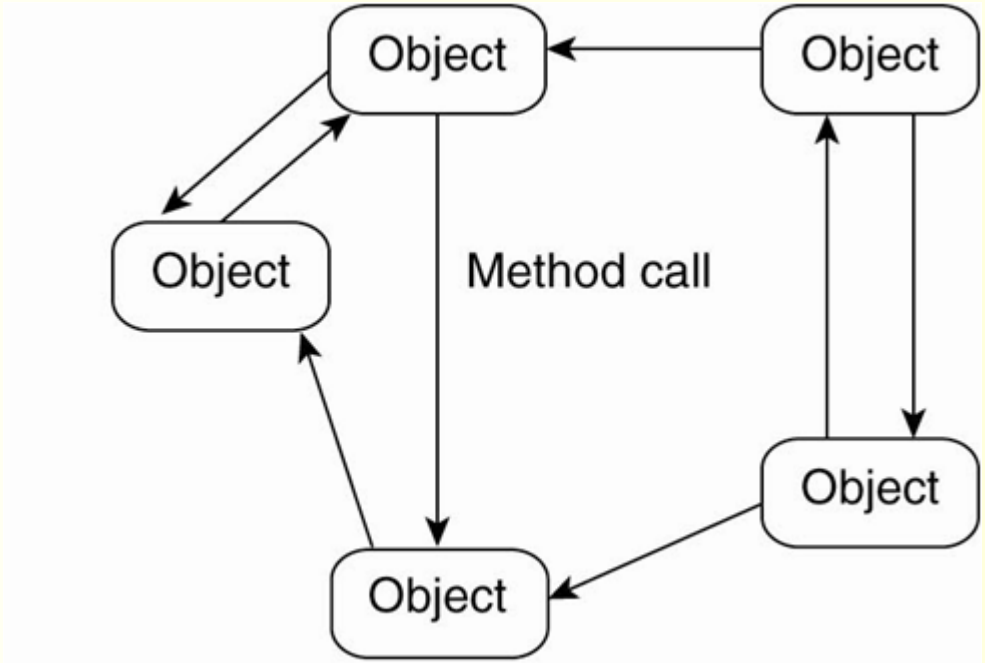
- refers to the logical organization of distributed systems into software components
- a component is a modular unit with well-defined, required and provided interfaces that is replaceable within its environment; can be replaced provided that we respect its interfaces
- a connector is a mechanism that mediates communication, coordination, or cooperation among components, e.g., facilities for RPC, message passing, or streaming multimedia data
- there are various architectural styles
  - Layered architectures
  - Object-based architectures
  - Data-centered architectures
  - Event-based architectures

- **Layered architectures**
  - **components are organized in a layered fashion where a component at layer L_i is allowed to call components at the underlying layer L_{i-1}, but not the other way around;**
  - **requests go down the hierarchy and results flow upward**
  - **e.g., network layers**



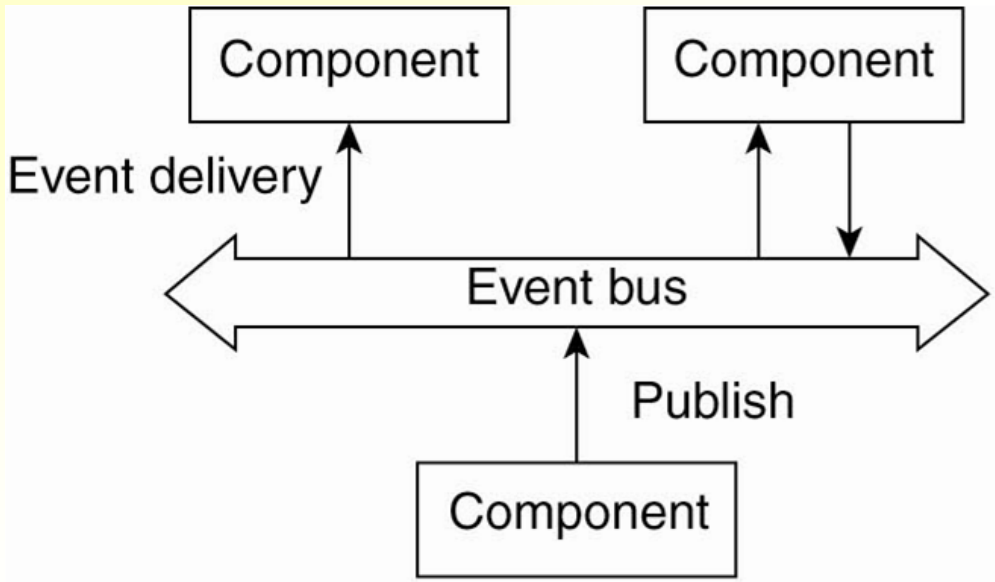
the layered architectural style

- **Object-based architectures**
  - **each object corresponds to a component and these components are connected through a remote procedure call mechanism (matches the client-server paradigm)**



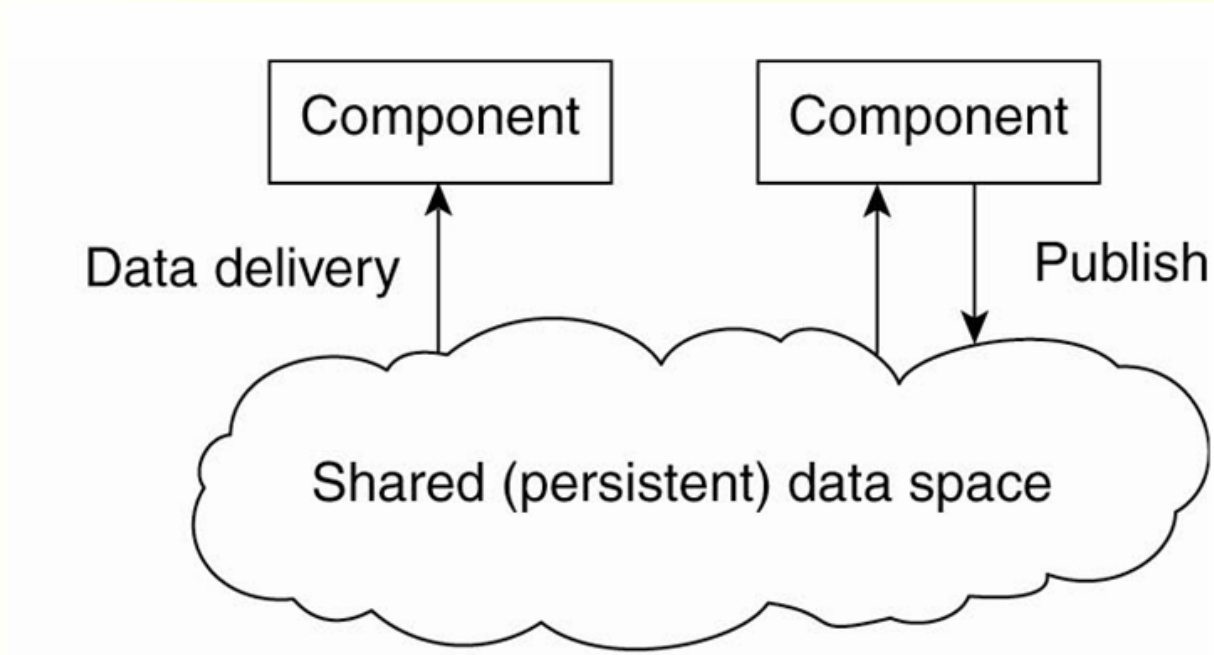
the object-based architectural style

- **Data-centered architectures**
  - **processes communicate through a common repository; e.g., a shared distributed file system**
- **Event-based architectures**
  - **processes communicate through the propagation of events (can also optionally carry data)**
  - **publish/subscribe systems**
  - **processes publish events and the middleware ensures that only those processes that subscribed to those events will receive them**
  - **processes are loosely coupled; no need of explicitly referring to each other**



the event-based architectural style

- **Shared data spaces**
  - **event-based architectures combined with data-centered architectures**
  - **processes are decoupled in time**



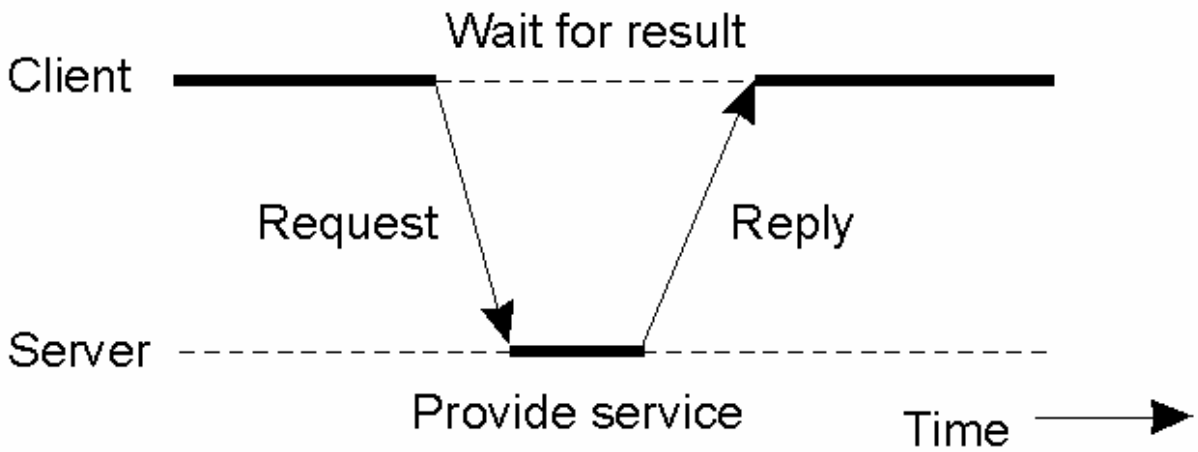
the shared data-space architectural style

2.2 System Architectures

- refers to the logical organization of distributed systems into software components or how are processes organized in a system; where do we place software components
- deciding on software components, their interaction, and their placement is what system architecture is all about
- can be centralized, decentralized or a hybrid

2.2.1 Centralized Architectures

- thinking in terms of clients requesting services from servers
- a server is a process implementing a specific service
- a client is a process that requests a service from a server by sending a request and waiting for a reply
- we have a request-reply behaviour

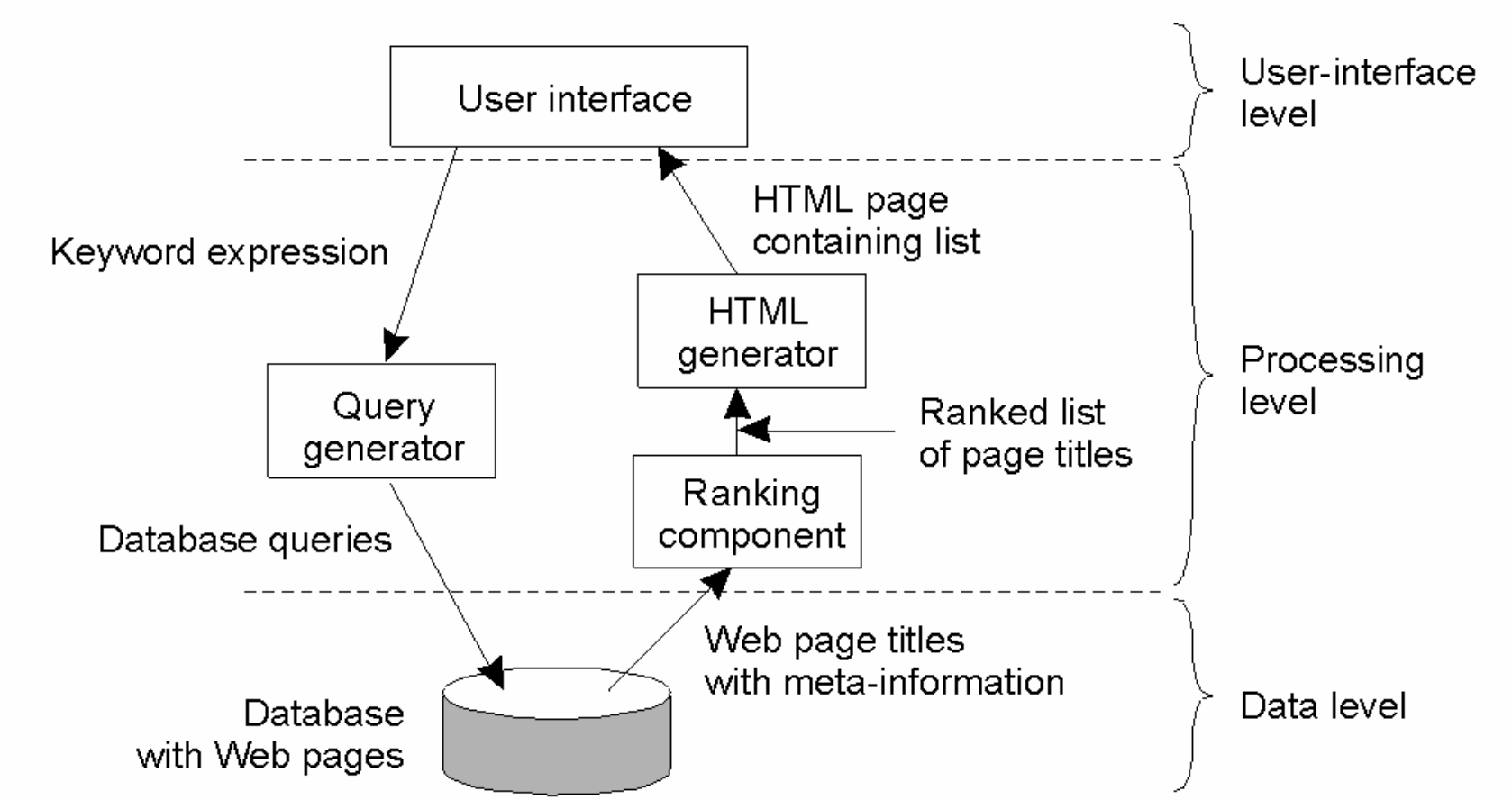


general interaction between a client and a server

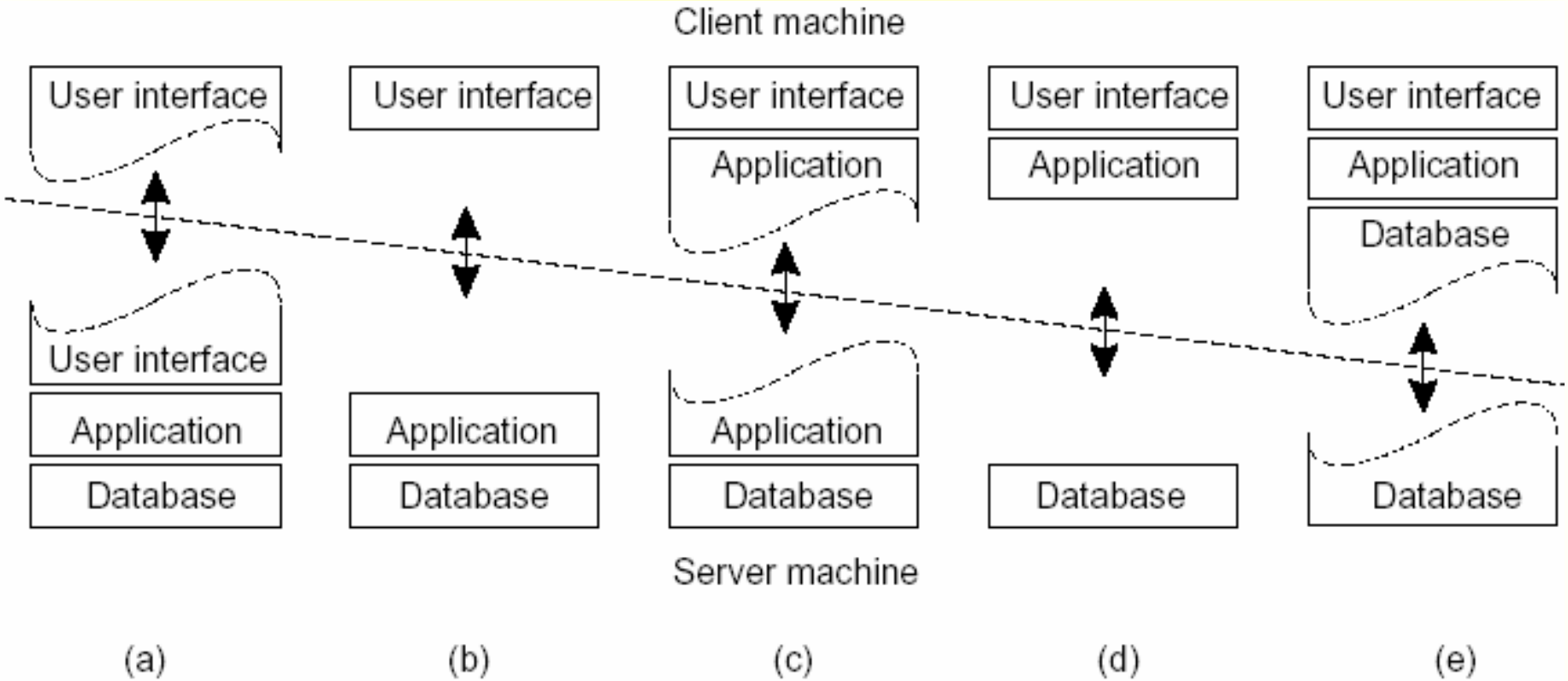
- communication between client and server can be
  - by a connectionless protocol if the underlying network is fairly reliable; efficient since there is no much overhead
    - but assuring reliability is difficult
    - we don’t also know the source of error; was the request or the reply lost, for instance
    - when messages are lost or corrupted let the client send the request again; applicable only for idempotent operations
      - an operation is idempotent if it can be repeated multiple times without harm; e.g., reading a record in a database
      - but, transferring an amount to a bank account is not idempotent
    - see later in Chapter 8 - Fault Tolerance
  - by a reliable connection-oriented protocol if the underlying network is unreliable
    - establishing and terminating connections is expensive

- **Application Layering**
  - **there are many controversies about the client-server model**
  - **e.g., no clear distinction between a client and a server; for instance a server for a distributed database may act as a client when it forwards requests to different file servers**
  - **three levels of distribution (following the layered architecture)**
    - **the user-interface level: implemented by clients and contains all that is required by a client; varying from a character-based screen to more advanced GUI-based interfaces (more on user interfaces in Chapter 3)**
    - **the processing level: contains the applications**
    - **the data level: contains the programs that maintain the actual data dealt with**

- **e.g., the general organization of an Internet search engine into three different layers**



- **Client-Server Architectures**
  - **how to physically distribute a client-server application across several machines**
  - **Multitiered Architectures**



Two-tiered architecture: alternative client-server organizations

(a) put only terminal-dependent part of the user interface on the client machine and let the applications remotely control the presentation

(b) put the entire user-interface software on the client side

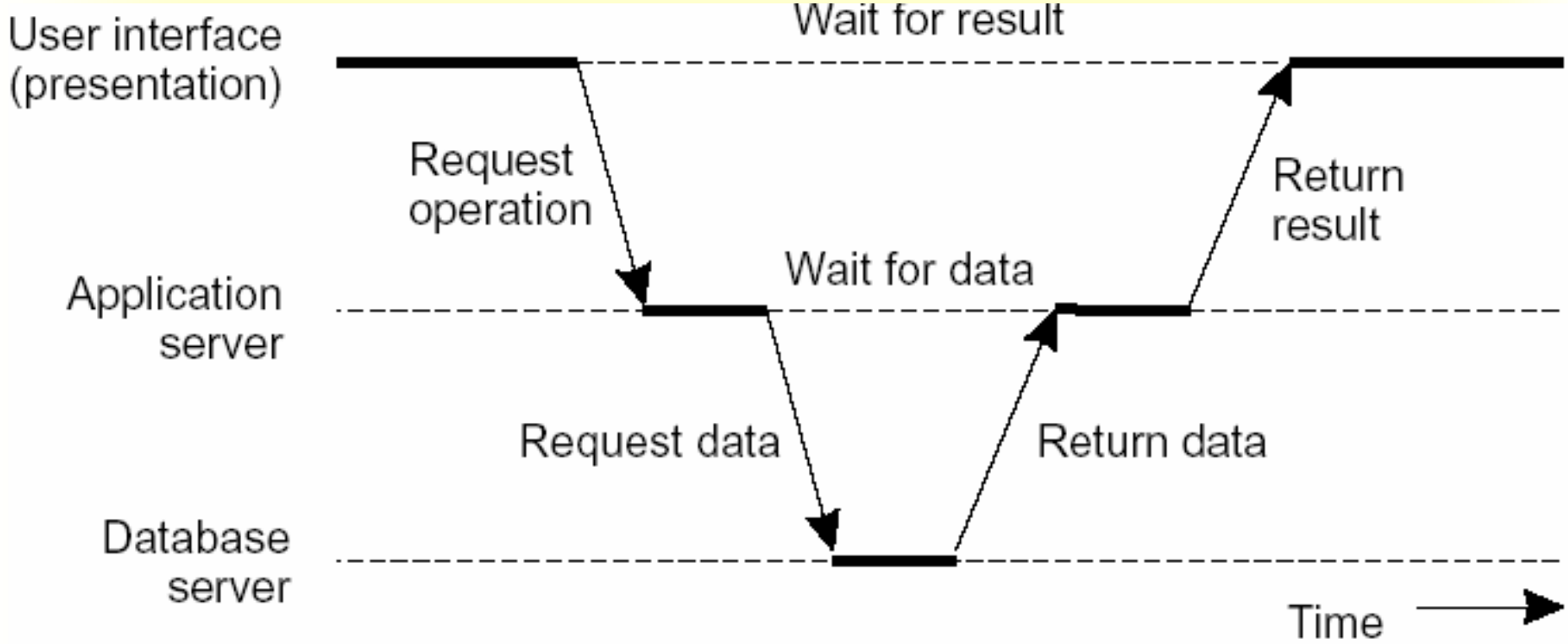
(c) move part of the application to the client, e.g. checking correctness in filling forms

(a) to (c) are for thin clients

(d) and (e) are for powerful client machines what are called fat clients (more popular)

(d) and (e) are difficult to manage since client-side software is distributed and is prone to error; it is also dependent on the client's platform such as operating system

- **a server may sometimes act as a client leading to a physically three-tiered architecture; an example is the organization of Web sites**



three tiered architecture: an example of a server acting as a client

2.2.2 Decentralized Architectures

- vertical distribution
  - refers to the ones discussed so far where the different tiers correspond directly with the logical organization of applications
  - place logically different components on different machines
- horizontal distribution
  - physically split up the client or the server into logically equivalent parts
  - an example is a peer-to-peer system where processes are equal and hence each process acts as a client and a server at the same time (servent)
  - read about the different approaches of peer-to-peer architecture - pages 44 - 51 and about Architectures versus Middleware - pages 54 - 66

- **another example is the horizontal distribution of a Web service**

